

Scope, boundaries and how the seams arose: a discovery guide for reviewers (r4.10)

- **For:** reviewers (people or models) who will judge the freeze proposal (FREEZE-PROPOSAL-R4.9.md). It assumes no earlier reading.
- **Status:** FOR REVIEW. It changes no code. It proposes eight candidate refinements (R-1 to R-8) that are simpler than some of r4.9's answers. They are not adopted.
- **Revision:** r4.10, 2026-10-06, by Claude.
- **Asked:** the owner asked to set out the scope and its boundaries; how the solutions we chose led to each current seam; how to solve them, including elegant solutions; and, looking back, whether even simpler solutions exist. The aim is to help reviewers *discover* these solutions, not only to read ours.

0. How to use this guide

Each boundary in §3 is laid out in the same order:

1. **What crosses it.**
2. **How we got here:** the chain of decisions and defects, in the order they happened.
3. **Questions:** try to answer them yourself first.
4. **Our answer (r4.9),** folded.
5. **Looking back:** a simpler answer we found while writing this guide, also folded, with **what would make it wrong.**

We would rather a reviewer find an answer simpler than ours than agree with ours. If you reach a different answer, record it in the markup table (§7) with your evidence: reproduced, measured, traced or opinion.

Identifiers: HF-nn is a defect found and fixed with a test (HARDENING.md); PD-nn is a decision; N-nn is a seam; DB-nn is a battery check; INV-n is a core invariant proposed in r4.9.

1. The scope in one sentence

A resident, confined process that watches declared parts of one host, records each change in a tamper-evident ledger, proves that it was watching while nothing changed, and stops for a person when it cannot see. It has no authority to act.

Inside the scope: observing (class 1a), recording, proving coverage, failing closed, and being judged before it runs.

Outside the scope: acting on anything; any network; any other host; judging other daemons; governing memory or reconciling state (PD-87); emergency authority (Break Glass); requests to external systems (the transport contract).

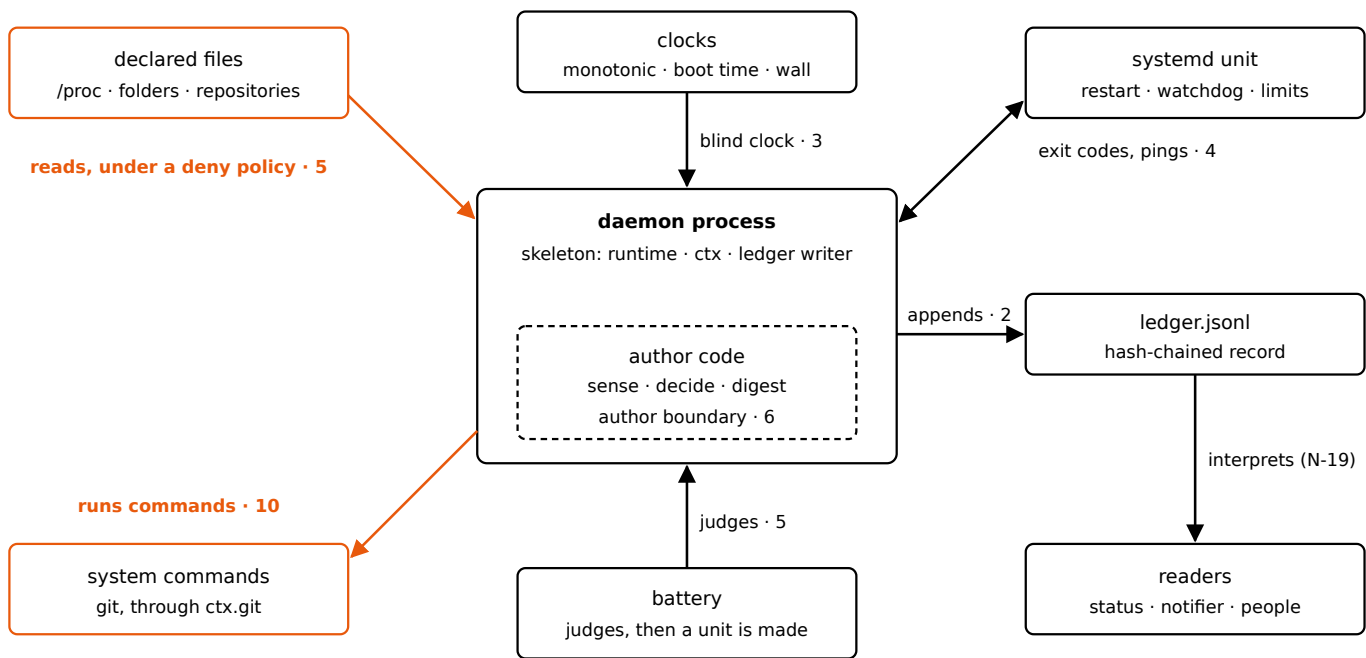
Every word of the sentence is load-bearing, and each one is a boundary:

Word in the sentence	The boundary it creates
"confined"	Host: what the process may read, write and run
"declared"	Author: what the author's code may do, through ctx
"proves that it was watching"	Time: blindness, heartbeats, clocks

Word in the sentence	The boundary it creates
"resident", "stops"	Service manager: exit codes, restarts, limits
"records ... tamper-evident"	Ledger and readers: who writes and who interprets
"is judged before it runs"	Activation: the battery, the unit, what actually runs
"no authority to act"	Extensions: everything above class 1a
(one pattern, used in more than one project)	Distribution: copies of the core

2. Where the defects sat

All 37 entries in HARDENING.md (HF-01 to HF-37), grouped by the boundary each one sat on:



Where the 37 recorded defects sat. Two edges (orange) carry 15: reading declared files under a deny policy (5) and running system commands (10). They include three of the four Critical defects (HF-01, HF-02, HF-24 on commands) and the fourth (HF-03) on the deny policy. Both edges can be shrunk by design (§3.1, §3.2). The other two of the 37 are a digest performance fix (HF-21) and a missing-evidence note (HF-23).

Boundary	Defects	Count	Critical
Running system commands	HF-01, 02, 08, 10, 11, 18, 19, 24, 29, 30	10	HF-01, HF-02, HF-24
Author code and ctx	HF-04, 05, 06, 09, 15, 33	6	—
Host policy (reads, writes, deny)	HF-03, 12, 14, 17, 36	5	HF-03
Judging and activation	HF-16, 20, 27, 35, 37	5	—
Service manager and limits	HF-07, 13, 22, 31	4	—
Time and blindness	HF-28, 32, 34	3	—
Ledger and lifecycle	HF-25, 26	2	—
Other	HF-21 (digest speed), HF-23 (missing evidence, not a defect)	2	—

Before reading on: what does this table suggest about where the design's complexity comes from?

3. The boundaries, one by one

3.1 Host: what the process may read, write and run (the deny policy)

What crosses it: file reads, directory listings, stat, writes to the output directory.

How we got here:

1. **r1.** The manifest declares reads and a deny list, and the skeleton adds a fixed `BASE_DENY` list (`~/ .ssh, ~/ .gnupg, ~/spark-core/data, ...`). Python checks every `ctx` path against both.
2. **r2.** Landlock is added as a kernel layer (AP-01). Landlock can grant a directory but cannot carve a denied folder out of it. A denied path inside a granted read becomes a recorded "gap" (PD-16), covered by the Python checks and, under `systemd`, by `InaccessiblePaths=`.
3. **HF-03 (Critical).** Daemon code emptied the policy object's deny list and read a base-denied path through exactly such a gap. Fixed by making the policy immutable.
4. **HF-17.** `ctx.stat` resolved a planted symlink before its policy check, so a link to a denied path stopped the daemon (78): a denial of service by anyone who could write to the watched folder. Fixed.
5. **HF-36 (r4.6).** Landlock granted execute wherever it granted read. Fixed.
6. **F-1 (r4.6, simulated).** A directory swapped for a symlink between the Python check and the open reaches a denied folder inside a granted read. The planned fix is a check after opening, which has to be designed around the audit hook.

Questions:

1. List every mechanism that enforces "do not read a denied path". How many are there, and do they all agree?
2. In which single case do they disagree?
3. What happens to F-1 if that case cannot occur?
4. How many shipped manifests (reference daemons, fixtures and the pilot's) need that case?

▼ Our answer (r4.9)

1. Four: the `ctx` check, the audit hook, Landlock and `InaccessiblePaths=`.
2. They disagree only for a gap: a denied path inside a granted read.
3. **Forbid gaps (PD-100).** The kernel's grant then *is* the policy, and F-1 closes by construction: a swapped symlink can only lead somewhere the kernel already refuses.
4. **None** (traced: `landlock.gaps()` returns `[]` for every shipped manifest).

▼ Looking back: R-1, remove the deny field

Once gaps are forbidden, a manifest's own deny list can only name paths that no declared read covers, and Landlock refuses those anyway. The field has no remaining effect, except as a validation constraint that `BASE_DENY` already provides. **Every shipped manifest, the pilot's included, has "deny": [].**

R-1: drop deny from the manifest. `BASE_DENY` stays as a validator constant ("no read may cover these"). The policy becomes one list, reads, and everything else is refused by the kernel.

What would make R-1 wrong: a real need to say "this repository, except that folder". The kernel cannot enforce that today, so the honest form of that need is two narrower reads. If a reviewer finds a case that two narrower reads cannot express, R-1 is wrong.

3.2 Host: running system commands

What crosses it: `ctx.run` and `ctx.git` start another program, with its own configuration, environment and filesystem reach.

How we got here:

1. **HF-01, HF-02 (Critical).** A Git alias ran a shell command, and `ctx.run(["git", ...])` bypassed the Git wrapper. Fixed by a hardened Git profile.
2. **HF-08.** Git refused the owner's repositories as "dubious ownership" under a dedicated user. Fixed with `safe.directory`.
3. **HF-10, HF-11.** Probing undeclared commands was not counted; the list of forbidden commands missed interpreter families and command runners.
4. **HF-18.** The CPU used by child processes was not counted.
5. **HF-19, HF-24 (Critical).** A repository's own `diff.external`, and `log.showSignature` with `gpg.program`, ran programs through `git diff` and `git log`. Fixed with more `-c` overrides and environment variables.
6. **HF-29.** Git output over the cap was treated as a failure.
7. **HF-30.** Six Git calls of up to 20 s each exactly filled the watchdog period.
8. **HF-36.** Execute had to be granted on system paths *because* commands exist.
9. **Planned:** PD-93 (pin each executable's digest) and PD-94 (an executable provenance check). Both exist only because commands exist.

Questions:

1. Which reference daemons declare a command? (Check `examples/*/manifest.json`.)
2. What does `git-watch` observe that it could not observe by reading files?
3. If the core ran no programs at all, which defects above could not have happened, and which planned work disappears?
4. Is running programs part of "watching", or a capability that only some watchers need?

▼ Our answer (r4.9)

r4.9 **kept commands in the core** and froze them, with the hardened Git profile, execute on system paths only (HF-36), and digest pinning (PD-93) planned. It did not question whether the core needs commands.

▼ Looking back: R-2, the core runs no programs

1. **One:** `git-watch`. Of the 12 shipped manifests, only one other declares a command: the hang test fixture (`sleep`, to test timeouts). The pilot declares none.
2. Branch and tag movements, and switches of HEAD, are files (`.git/HEAD`, `.git/refs/`, `.git/packed-refs`), readable with `read_text` and `list_dir`. Only "is the worktree dirty" needs `git status`.
3. **Ten defects** (three of them Critical) sit on this edge, and PD-93, PD-94, `proc.py` (220 lines) and the Git hardening profile exist only for it. Without commands, Landlock need grant execute nowhere once the daemon has started. *This needs a test: a C-extension import under a domain that grants no execute.*
4. **A capability.** Most watchers read `/proc`, folders or files.

R-2: the frozen core runs no programs. Running commands becomes the first extension, a "command profile" that adds `commands`, `ctx.run`, `ctx.git`, `proc.py`, execute on system paths, PD-93 and PD-94. Adding it later is purely additive. `git-watch` moves to that profile, or splits into a core refs watcher (file reads only) and a worktree watcher in the profile.

What would make R-2 wrong: if most planned daemons need commands. The Repository Observer reads Git, so a reviewer should count the planned daemons and their needs. If commands are the common case, they belong in the core, and the hardening stays frozen with them.

3.3 Author: what the author's code may do

What crosses it: the three functions the author writes (sense, decide, digest) and every value they pass back through `ctx`.

How we got here:

1. **HF-04, HF-05, HF-06.** Three escapes from the static purity check (frame introspection, aliasing open, dynamic attribute access).
2. **HF-09.** `raise SystemExit(0)` in author code ended the process with no stop record.
3. **HF-15.** A nanosecond timestamp exceeded the canonical integer range.
4. **HF-33 (r4.3).** A capped directory listing was diffed as if it were the whole folder.
5. **PD-83.** Truncation will always raise.

Questions:

1. If the purity check is bypassed, what stops the daemon from reading a denied file or opening a socket?
2. Should a new escape from the purity check be a safety defect (HF-*nn*) or a diagnostic bug?
3. Is there any value `ctx` returns that an author could mistake for complete when it is not?

▼ Our answer (r4.9)

1. The kernel: Landlock and the unit. DB-17 proves that the kernel blocks them with the audit hook off.
2. **A diagnostic bug (PD-101).** Purity and the audit hook stay as fast, readable author feedback and a tripwire. The safety claim rests on the kernel (INV-1).
3. Today, yes: `ctx.run`, `ctx.git` and `ctx.list_dir` return truncation as a value. **PD-83 makes truncation always raise**, removing seam N-10.

▼ Looking back

The author API is already minimal: three functions, with `decide` pure. We found nothing simpler. One consequence of PD-101 is worth stating: **stop hardening the purity check**. HF-04 to HF-06 show that a determined author will always find another route, and the kernel already contains what they reach. Effort on the purity check after the freeze is lint work, not safety work.

3.4 Time: proving the daemon was watching

What crosses it: the question "has this daemon been seeing anything?", asked by the daemon itself, by its next start, by the battery and by readers.

How we got here:

1. **r3.5, PD-63.** A blind clock: no accepted cycle within `T` means `SENSE_BLIND` and exit 78.
2. **HF-28.** The unit restarted exit 78 forever. Fixed with `RestartPreventExitStatus=`.
3. **HF-32 (r3.9).** The clock restarted at zero with each process, so a restart loop slower than the start limit hid blindness forever.
4. **r4.0, PD-70.** Inherited blindness is read from the ledger, and a heartbeat is written every `T/2`.
5. **HF-34 (r4.5).** A backward wall-clock step made every restart inherit zero. Fixed with `CLOCK_BOOTTIME` and `boot_id`, with a worst-case assumption across reboots.
6. **HF-35.** The battery passed a daemon that never saw anything. Fixed with DB-20.
7. **Seams that remained:**
 - N-5: an external staleness checker would compare times on the wall clock;

- N-21: the gap between a stop and the next start reads as quiet;
- N-22: a unit skipped by its start condition writes nothing.

Questions:

1. In how many places is "how long has it been blind?" computed, or planned to be?
2. What facts does each place need? Are they the same facts?
3. BM-3 says an independent monitor, not the blind component, should make the switch. Our daemon stops itself. Which is right, or are both needed?

▼ Our answer (r4.9)

1. Three: the running process (`_Blind`), each start (`_inherited_blindness`), and a planned external checker. Every external reader adds one more.
2. r4.9 adds a status command (PD-105) that derives observing, quiet, blind, stopped, not watching and coverage from the ledger, on `CLOCK_BOOTTIME` within a boot. That closes N-5, N-21 and N-22.
3. r4.9 keeps both: the daemon stops itself (fail closed works even with no monitor), and status is the outside view.

▼ Looking back: R-4, one blindness function

Question 2: the same facts. Start-up and status both need the latest evidence of an accepted cycle, its boot stamp, the current boot time and boot id, the wall clock and T. They differ only in who calls them.

R-4: one pure function, `blind_for(records, now, T)`, which returns the blind duration and the clock basis. Start-up recovery and status both call it. It is written once, tested once, and cannot drift.

This leaves a real trade-off for the reviewer. U-7 asks for a *diverse* implementation of each monitor's critical check, and r4.9 made the reader independent of the skeleton for that reason. R-4 puts one interpretation in both. Our suggested split is that **verifying bytes and hashes** stays diverse (two implementations, DB-22), while **interpreting** what the records mean is single (one function), because two interpretations of one rule will drift.

Looking further: PD-01.3's Sentinel (`SENSE_DEGRADED` at T/2) may need no new record type at all. Heartbeats already carry mode: `blind` and `blind_ms`, so status can derive "degraded" from them. A reviewer should check whether the Sentinel is just a status state.

What would make R-4 wrong: if U-7 requires the start-up path and the external path to be independent *interpretations*, for example because a defect in the shared function would blind both at once. The counterweight is that a shared function is tested by both callers.

3.5 Service manager: exit codes, restarts and limits

What crosses it: exit codes, watchdog pings, and the unit's timeouts and start limits.

How we got here:

1. **HF-07.** The unit's system call filter blocked Landlock, so the daemon could never start as installed.
2. **HF-13, HF-22.** Unvalidated text reached the unit file.
3. **HF-28.** Fail-closed exits were restarted.
4. **HF-30.** A worst-case cycle filled the watchdog period.
5. **HF-31.** The stop timeout was shorter than a cycle.
6. **U-8, PD-69.** Limits are checked against each other, in a budget table (DB-21).
7. **U-6.** Start-up time grows with the ledger, against a fixed start timeout.

8. **r4.5.** The exit code 78 split was adopted, creating N-12 (four consumers of one table). r4.9 withdrew it (PD-102).

Questions:

1. List the manifest's time numbers and the unit's time numbers. Which of them must satisfy a relation with another?
2. For each such relation, is either number *derived* today, or are both declared?
3. HF-28, HF-30, HF-31 and HF-32 are all broken relations between limits. Could a different choice of what is declared have made them impossible?

▼ Our answer (r4.9)

1. Declared: the interval, `watchdog_seconds`, `step_timeout_seconds` and `blind_limit_seconds` (T). Derived already: the heartbeat ($T/2$), the stop timeout (`watchdog + 10 s`, after HF-31), and the start timeout ($\max(60 \text{ s}, \text{watchdog})$).
2. r4.9 keeps the declarations and checks the relations in DB-21's budget table.
3. Exit codes encode only the restart class (PD-102); the reason is in the ledger.

▼ Looking back: R-6, declare a cycle budget and derive the rest

Question 3: yes. HF-30 happened because the watchdog and the worst-case cycle are both declared, and nothing ties them together. The runtime knows each `ctx` call's deadline but not the cycle's.

R-6: the manifest declares one `cycle_budget_seconds`, and the runtime enforces it. Each `ctx` call gets at most the time left in the cycle; past the budget, the call raises and the cycle fails, which counts toward blindness, as any failed cycle does. Then:

- `watchdog = 2 × budget + a margin` (derived);
- `stop timeout = watchdog + 10 s` (derived, as today);
- `step_timeout_seconds` disappears;
- the HF-30 class becomes impossible by construction, and DB-21 shrinks to the relations that remain (T against the interval; start-up against the ledger).

Principle: when two numbers must satisfy a rule, declare one and derive the other.

What would make R-6 wrong: pure-Python work in sense or decide that never calls `ctx` cannot be interrupted by the budget. The watchdog still catches it (a kill, then a restart, which counts toward blindness), so the bound holds, but less gracefully. A reviewer should judge whether that residue is acceptable.

3.6 Ledger and readers: who writes, and who interprets

What crosses it: `ledger.jsonl`, read by the next start, the battery, verify, the pilot's notifier and people.

How we got here:

1. **HF-25, HF-26.** Lifecycle order: a rejected duplicate launch deleted the running instance's files, and a torn first write made a ledger the battery rejected.
2. **Start-up verifies the whole chain** and refuses to start on any corrupt line (DB-07). This comes from the ledger recovery rules (PD-32).
3. **U-6.** Start-up verification is linear: about 28,000 records a second here (*evidence/r4.9/*). For a chatty daemon, it reaches the 60 s start timeout within months.
4. **N-19.** The pilot's notifier reads the ledger without verifying it, works out "restarted" from a payload flag, and tracks a byte offset, which segmentation would break.

5. **r4.9**: an independent reader, status, positions by seq, and a segmentation rule fixed now and built later (PD-105, PD-106).

Questions:

1. What does a start actually need from the ledger in order to continue it correctly?
2. What does verifying the *whole* chain at every start protect against? An attacker who can write the file can recompute the chain (SQLITE-LEDGER-REVIEW.md point 4), so which threat is left?
3. What is better for an observer: stopping when an old line is found corrupt, or carrying on and reporting the corruption?

▼ Our answer (r4.9)

r4.9 keeps the full scan at start and fixes U-6 with a segmentation rule: segments are built when DB-21 projects that start-up will pass half the start timeout. Readers position by seq.

▼ Looking back: R-5, start-up verifies only the tail

1. The tail: the last complete record's seq and hash, a check that it links to the record before it, a check for a torn tail, and the latest evidence of an accepted cycle. That evidence is in the last few records, because a heartbeat, start or stop record carries it.
2. **Accidental corruption of old lines**, for example a disk error. That is worth detecting, but it says nothing about whether the daemon can watch *now*.
3. Carrying on. Refusing to start turns an old disk error into an observation outage: the daemon goes blind *because* of a problem in its past.

R-5: start-up verifies only the tail, in constant time; full verification is verify and status, run on a schedule from outside. Corruption is then reported as a status state (ledger corrupt at seq n), and the daemon keeps observing, appending to a chain whose break stays detectable forever. U-6 disappears for start-up, segmentation is needed only for disk space, and PD-106 shrinks to "readers position by seq".

What would make R-5 wrong: it changes INV-5 and the recovery rule the pattern took from the ledger specification it follows (PD-32: corruption means refusing to start and changing nothing). If an owner or another component relies on "a running daemon implies an intact chain", R-5 breaks that. This is a decision for the owner, not a refinement.

3.7 Activation: is what runs what was judged?

What crosses it: the battery's verdict, the generated unit, and the code the unit starts.

How we got here:

1. **HF-16, HF-35.** The battery passed daemons that failed every cycle, or never saw anything.
2. **HF-27.** A PASS did not name the code it passed.
3. **HF-37 (r4.7).** precheck and battery bound different copies of the manifest, so one passed what the other failed.
4. **PD-40** plans a register of approved digests; **PD-50** says no zero-touch activation.
5. **The pilot** automated activation: the battery runs on the host, and the unit is installed on PASS. Nothing compares what runs later with what passed (N-20).
6. **r4.9:** unit --report pins the digests and host facts of a PASS; the runtime refuses a mismatch (PD-104).

Questions:

1. How many tools judge a daemon today (validate, precheck, battery)? What did HF-37 show about having more than one?
2. In r4.9, two steps must be done in the right order: run the battery, then generate the unit from its report. Could the order be made impossible to get wrong?

▼ Our answer (r4.9)

PD-104: the unit generator accepts a PASS report, checks it against the files and the host, and writes the digests into ExecStart=. The runtime refuses a mismatch.

▼ Looking back: R-3 and R-7, one judge and one path to a unit

R-3: only the battery can emit an installable unit. On PASS, battery --emit-unit writes the pinned unit; on anything else it writes nothing. unit remains as a preview, marked unpinned, which the runtime refuses to run outside test mode. There is then no order to get wrong, and no report file to pass between steps.

R-7: one judge. precheck is the battery with a subset of checks, on the same code path and with the same workspace binding, and validate is the battery's static first stage. HF-37 is the lesson: two judges of one thing drift.

What would make R-3 wrong: an operator who must generate a unit on a machine other than the one that ran the battery. Under G-5, though, a PASS on another host should not activate this one, so that case is the one R-3 is meant to forbid.

3.8 Extensions and distribution

What crosses it: everything above class 1a (Break Glass, the Blind Forester, the transport contract, the supervisor/worker split, SQLite ledgers), and copies of the core in other projects.

How we got here:

- The extension designs were made early, so the core could not paint itself into a corner. They then added fields and seams to the core's plans (pacing, the consequence field, an inbound acknowledgement): N-1, N-3, N-6 to N-9, N-11, N-14 and N-15 to N-18.
- **The pilot required a rename:** its CI forbids the word "spark" in paths. The rename changed the contract's hash (2d00dda2... here, f7d5ee8a... there), so the two copies can no longer be compared mechanically. Changes went across by hand in both directions (N-23).

Questions:

1. Which extension *needs* a change to a frozen contract? Which could arrive as a separate process, or as an optional field?
2. Why do the two copies have different contract hashes when their rules are the same?

▼ Our answer (r4.9)

1. None needs a change (PD-99). Each arrives beside the core: as a separate process, an internal split (PD-72), or an additive record type or field.
2. The rename (package, environment variables, unit prefix). r4.9 makes the flow one way: changes land in the core first and are re-vendored with a mechanical rename (PD-108).

▼ Looking back: R-8, a neutral core name

Question 2: the hash differs only because names inside the contract differ.

R-8: give the core a name that belongs to no project, and make host-specific names

generator options: the unit prefix (spark-daemon- or firstborn-), the environment variable prefix and the command name. A vendored copy is then byte-identical, and parity is one comparison: **the two contract hashes are equal**. N-23 becomes a check, not a discipline.

What would make R-8 wrong: the cost of a one-time rename in this repository (imports, environment variables, documentation), against a check that never drifts. A reviewer should weigh whether more than one or two projects will ever vendor the core.

4. What the chains have in common

Before reading on: look back at the eight "How we got here" lists. What do the fixes that *created* new seams have in common?

▼ Six lessons

#	Lesson	Seen in
LS-1	When a fix adds a new place where a fact is kept, a seam appears between the old place and the new one. Keep each fact once, and derive the rest	The blind clock (process, ledger, clock, readers); consumers re-deriving the ledger's meaning (N-19)
LS-2	When a policy language says more than its enforcer can, the difference becomes a race. Shrink the language to the enforcer	The deny policy, F-1
LS-3	When two numbers must satisfy a rule, declare one and derive the other	HF-28, HF-30, HF-31; R-6
LS-4	Two judges of one thing drift	HF-37; four ledger readers
LS-5	A component cannot report its own absence. Liveness is judged from outside, from facts the component wrote, on a clock both sides share	The watchdog is not seeing (S-1); a digest cannot say "dead"; N-21, N-22
LS-6	Every capability carries its whole surface into the core. Ask whether watching needs it, or only some watchers do	Commands (10 defects); plans for future classes leaking fields into the core

5. Candidate refinements to r4.9

#	Refinement	Simpler than	Breaks	Removes	Owner?
R-1	Drop the manifest's deny field; BASE_DENY stays as a validator constant	PD-100	No shipped manifest (all have []); the field goes in 4.0.0	A list and its checks	No
R-2	The core runs no programs; commands become the first extension	Freezing commands	git-watch moves to the extension	proc.py, the Git profile, PD-93, PD-94, execute rights; the edge with 10 defects	No, but needs the count of planned daemons that run commands
R-3	Only the battery emits an installable unit	PD-104	The pilot's installer calls battery --emit-unit instead of unit	A step that can be done in the wrong order	No
R-4	One blindness function for start-up and status	Two implementations	Nothing	A seam between two interpretations	Trade-off with U-7
R-5	Start-up verifies the tail only; full verification is scheduled outside	PD-106's segmentation trigger	INV-5's "refuse to start", from PD-32	U-6 for start-up; a reason to go blind	Yes

#	Refinement	Simpler than	Breaks	Removes	Owner?
R-6	Declare cycle_budget_seconds; derive the watchdog	DB-21's watchdog relations	watchdog_seconds and step_timeout_seconds leave the manifest (4.0.0)	The HF-30 class	No
R-7	precheck and validate are subsets of the battery, on one code path	Three judges	Nothing visible	The HF-37 class	No
R-8	A neutral core name; host names become generator options	PD-108's mechanical rename	A one-time rename here	Hand-porting; N-23	No

6. The core, if every refinement were taken

	r4.8 (built)	r4.9 (proposed)	With R-1 to R-8
Manifest fields (top level)	18	18	15 (removes commands, deny, watchdog_seconds, step_timeout_seconds; adds cycle_budget_seconds)
ctx methods	8	8	6 (removes run, git)
Places that enforce a read policy	4, which disagree on gaps	4, which agree	1 that decides (the kernel) , plus diagnostics
Places that compute blindness	2, plus 1 planned	3	1 function, 2 callers
Judges of a daemon	3	3	1, with levels
Ways to get an installable unit	1 (unpinned)	1 (pinned, two ordered steps)	1 (the battery, on PASS)
Start-up cost against ledger size	Linear	Linear, then segmented	Constant (if R-5)
Copies comparable mechanically	No	No	Yes (same contract hash)

7. Questions for the reviewer, and the markup table

- For each boundary in §3: did you reach our answer, a simpler one, or a different one? Say which, with evidence.
- R-2:** of the daemons planned for this project, how many need to run a program? If most do, R-2 is wrong.
- R-5:** should an observer ever stop *because of its past*? This is the owner's decision; give your reasoning.
- R-4 against U-7:** where is the line between "diverse verification" and "one interpretation"?
- LS-1 to LS-6:** is there a seventh lesson we missed? Where else in the design does it apply?
- What else, looking back, could be simpler still?

#	Item	Reached ours / simpler / different	Evidence type	Notes
1	§3.1 host: deny policy (R-1)			
2	§3.2 host: commands (R-2)			
3	§3.3 author			
4	§3.4 time (R-4)			
5	§3.5 service manager (R-6)			
6	§3.6 ledger and readers (R-5)			

#	Item	Reached ours / simpler / different	Evidence type	Notes
7	§3.7 activation (R-3, R-7)			
8	§3.8 extensions and distribution (R-8)			
9	§4 lessons			